Algorithm Cheat Sheet — All Templates + Java Syntax

Conventions, Skeleton & Core API

Generic Test Harness

```java
class Question {
    // —— PROBLEM-SOLVING PROCESS
    // 1. INPUT / OUTPUT : types, sizes, ranges; what to return; examples.
    // 2. HIGH-LEVEL ALGO : pick the pattern (two-ptr / BFS / DP / ...); state the idea in 1 line.
    // 3. IMPLEMENTATION : data structures, loop/recursion, invariants.
    // 4. EDGE CASES : empty, size 0/1, duplicates, negatives, overflow, all-same, not-found.
    // 5. COMPLEXITY : time & space; is it within limits?
    // 6. OPTIMIZATION : reduce time/space (hashing, two-pointer, rolling array, pruning, early exit).

    private int data;
    public Question(int data) {
        this.data = data;
    }
    public void solve() {
        // TODO
    }
}
public class Solution {
    public static void main(String[] args) {
        Question q = new Question(0);
        q.solve();
    }
}
```

Naming Conventions (enforced across all template files)

Common Java API Cheat-Sheet

```java
Map<Integer, Integer> map = new HashMap<>();        // key → value
Map<Integer, List<Integer>> adj = new HashMap<>();  // adjacency / buckets
Set<Integer> set = new HashSet<>();                 // membership
List<Integer> list = new ArrayList<>();             // dynamic array
Deque<Integer> stack = new ArrayDeque<>();          // STACK: push / pop / peek
Deque<Integer> queue = new ArrayDeque<>();          // QUEUE: offer / poll / peek
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
TreeMap<Integer, Integer> tmap = new TreeMap<>();   // sorted keys
TreeSet<Integer> tset = new TreeSet<>();            // sorted set
Random rand = new Random();
int[] arr = new int[n];
boolean[] seen = new boolean[n];
int[][] grid = new int[m][n];
int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};   // DOWN,UP,RIGHT,LEFT
```

```java
Integer.MAX_VALUE;   // 2147483647
Integer.MIN_VALUE;   // -2147483648
Long.MAX_VALUE;
int k = i + (j - i) / 2;    // avoid overflow
Integer.compare(a, b);
Double.compare(a, b);
```

```java
ch - 'a';                 // 0..25 (lowercase bucket index)
ch - '0';                 // 0..9 (digit value)
(char) ('a' + i);
num = num * 10 + (ch - '0');
Integer.parseInt("123");
String.valueOf(123);
Character.isDigit(ch);
Character.isLetter(ch);
Character.isLetterOrDigit(ch);
Character.isWhitespace(ch);
Character.toLowerCase(ch);
Character.toUpperCase(ch);
```

```java
s.length();  s.charAt(i);  s.substring(i, j);
s.toCharArray();  s.split(" ");
s.equals(t);  s.compareTo(t);
s.indexOf("ab");  s.isEmpty();
s.startsWith("ab");  s.endsWith("z");
s.contains("x");
s.toLowerCase();  s.toUpperCase();
s.trim();  s.replace('a', 'b');
s.repeat(3);
StringBuilder sb = new StringBuilder();
sb.append(x);  sb.reverse();
sb.toString();  sb.insert(0, x);
sb.charAt(i);  sb.setCharAt(i, c);
sb.deleteCharAt(i);
sb.length();
String.join(",", list);
```

```java
Arrays.sort(arr);
Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
Arrays.fill(dp, Integer.MAX_VALUE);
Arrays.equals(window, need);
Arrays.copyOf(arr, len);
Arrays.copyOfRange(arr, i, j);
System.arraycopy(src, srcPos, dst, dstPos, len);
Arrays.asList(1, 2, 3);
int sum = Arrays.stream(arr).sum();
int max = Arrays.stream(arr).max().getAsInt();
```

```java
Math.max(a, b);  Math.min(a, b);
Math.abs(x);
Math.pow(a, b);  Math.sqrt(x);
Math.ceil(x);  Math.floor(x);
```

```java
map.put(k, v);
map.getOrDefault(k, 0);
map.merge(k, 1, Integer::sum);
map.computeIfAbsent(k, x -> new ArrayList<>()).add(v);
map.putIfAbsent(k, v);
map.containsKey(k);  map.remove(k);
for (Map.Entry<Integer, Integer> e : map.entrySet()) { e.getKey(); e.getValue(); }
map.keySet();  map.values();
set.add(x);  set.contains(x);
set.remove(x);
list.add(x);  list.get(i);  list.set(i, x);
list.remove(list.size() - 1);
list.size();  list.isEmpty();
list.addAll(other);
Collections.sort(list);
Collections.reverse(list);
```

```java
tmap.firstKey();  tmap.lastKey();
tmap.floorKey(k);    // largest key <= k
tmap.ceilingKey(k);  // smallest key >= k
tmap.lowerKey(k);    // largest key < k
tmap.higherKey(k);   // smallest key > k
tset.floor(x);  tset.ceiling(x);
tset.first();  tset.last();
```

```java
// Deque as STACK (LIFO)
stack.push(x);
int top = stack.pop();
int look = stack.peek();
stack.isEmpty();
// Deque as QUEUE (FIFO)
queue.offer(x);
int head = queue.poll();
queue.isEmpty();
deque.offerLast(x);  deque.pollLast();
deque.peekLast();
deque.offerFirst(x);  deque.pollFirst();
deque.peekFirst();
pq.offer(x);  int best = pq.poll();
int look = pq.peek();  pq.size();
pq.remove(x);
```

Template Index

Two Pointers

Pattern 1 — Opposite Two Pointers

Variables: i = left pointer · j = right pointer · scan range [i, j] closed

Pseudocode:

```
set i to the left end, j to the right end
while the pointers haven't crossed:
    if the current pair hits the target:
        record it, then step both ends inward
    else if the value is too small:
        move the left end right to grow it
    else (too large):
        move the right end left to shrink it
```

```java
int i = 0, j = n - 1;
while (i < j) {
    if (condition == target) {
        // record
        i++; j--;
    } else if (tooSmall) {
        i++;
    } else {
        j--;
    }
}
```

Sliding Window

The Three Templates

Variables: i = left edge (inclusive) · j = right edge / loop variable (inclusive), window = [i, j], size = j − i + 1 · k = target window size · result = answer (max length, or min length seeded to MAX)

Pseudocode:

```
FIXED window of size k:
    i = 0
    for j = 0..n-1:
        add nums[j] to window state
        if window size (j - i + 1) == k:
            record the window (it is exactly size k)
            remove nums[i] from state; i++

MAX variable window (longest valid):
    i = 0; result = 0
    for j = 0..n-1:
        add nums[j] to state
        while window violates the constraint:
            remove nums[i] from state; i++
        result = max(result, j - i + 1)

MIN variable window (shortest valid):
    i = 0; result = infinity
    for j = 0..n-1:
        add nums[j] to state
        while window is valid:
            result = min(result, j - i + 1)
            remove nums[i] from state; i++
```

Frequency Map Trick (used by 567 and 76)

Prefix Sum + HashMap

Core Template

Variables: map = prefixSum→count or index seen so far · sum = running prefix sum · lookup = the complement prefix we search for · i = current scan position

Pseudocode:

```
create empty map of prefix -> (count or index)
seed map with prefix 0 (count=1 for counting, index=-1 for length)
sum = 0
for each index i:
    add nums[i] to sum
    lookup = transform(sum)
    use map.get(lookup) to update result
    store sum (or sumMod) into map with count or index
```

```java
Map<Long, Integer> map = new HashMap<>();
map.put(0L, ???);
long sum = 0;
for (int i = 0; i < nums.length; i++) {
    sum += nums[i];
    long lookup = transform(sum);
    map.put(storeKey(sum), storeValue(...));
}
```

Binary Search

The Only Template — [i, j], find first k with condition(k)

Variables: [i, j] = half-open search range (answer lives in here) · k = midpoint · condition(k) = monotone test, false…false then TRUE…true; answer is the first k it holds

Pseudocode:

```
i = 0, j = length          # half-open [i, j)
while range non-empty (i < j):
    k = midpoint of i and j
    if condition(k) is false: answer is to the right, set i = k+1
    else: k might be the answer, set j = k
```

```java
int i = 0, j = nums.length;
while (i < j) {
    int k = i + (j - i) / 2;
    if (!condition(k)) {
        i = k + 1;
    } else {
        j = k;
    }
}
```

How to Choose

Sorting

Template 1 — Merge Sort

Variables: [start, end] = half-open range being sorted · mid = split point · temp = scratch buffer · i = left scan pointer [start, mid) · j = right scan pointer [mid, end) · k = write pointer into temp

Pseudocode:

```
mergeSort(nums, start, end):
    if range has 0 or 1 elements: return   // already sorted
    mid = midpoint of [start, end)
    mergeSort left half [start, mid)
    mergeSort right half [mid, end)
    merge the two sorted halves
```

```java
public void mergeSort(int[] nums) {
    mergeSort(nums, 0, nums.length, new int[nums.length]);
}
private void mergeSort(int[] nums, int start, int end, int[] temp) {
    if (end - start <= 1) return;
    int mid = start + (end - start) / 2;
    mergeSort(nums, start, mid, temp);
    mergeSort(nums, mid, end, temp);
    merge(nums, start, mid, end, temp);
}
```

Template 2 — Quick Sort (3-way Dutch Flag Partition)

Template 3 — Quick Select (partial sort — k-th element)

```java
private int quickSelect(int[] nums, int start, int end, int target) {
    ...
}
```

Template 4 — Counting Sort

Variables: min/max = value range bounds · buckets[v−min] = count of value v · i = write pointer back into arr · v = current value index into buckets

```java
public void countingSort(int[] arr) {
    if (arr.length == 0) return;
    int min = Arrays.stream(arr).min().getAsInt();
    int max = Arrays.stream(arr).max().getAsInt();
    int[] buckets = new int[max - min + 1];
    for (int x : arr) buckets[x - min]++;
    int i = 0;
    for (int v = 0; v < buckets.length; v++)
        while (buckets[v]-- > 0) arr[i++] = v + min;
}
```

Linked List

When to Use — Signal → Pattern

All Four Templates

Variables: sentinel = dummy node before head · previous = last kept node · current = node under inspection · next = saved successor · slow/fast = 1×/2× walkers · a/b = the two input lists

String

Core Idioms

Variables: count = frequency vector (index = char/digit bucket) · ch = 'a'/'0' bucket index · num = number built so far · builder = encoded/result string · buckets = lists indexed by frequency · expand(i, j) = palindrome length from a center

Stack / Queue / Heap

All Templates

Variables: stack = LIFO ArrayDeque (holds indices for monotone variants) · queue = monotone deque (front = window answer) · minPQ/maxPQ = priority queue (heap) · maxHeap = lower half, minHeap = upper half for median

BFS (Tree)

The Template — Level-Aware BFS

Variables: queue = nodes waiting to be processed · size = level snapshot (count of nodes in the current level) · level = current depth counter · node = node polled this step · i = index within the level

Pseudocode:

```
make an empty queue
put root in the queue
level = 0
while the queue is not empty:
    size = how many nodes are in the queue right now (one whole level)
    level = level + 1
    repeat size times (index i = 0..size-1):
        node = take the front of the queue
        process node (level known; i==0 -> first, i==size-1 -> last)
        if node has a left child, add it to the queue
        if node has a right child, add it to the queue
```

When to Use — Signal → Pattern

The One Rule — Snapshot Level Size First

DFS / Backtracking

When to Use — Signal → Pattern

All Templates

Graph

Complexity Legend

```
BFS / DFS            O(V + E)     visit each vertex and edge once
Topo sort (Kahn)     O(V + E)     each node enqueued once, each edge relaxed once
Union-Find           ~O(n·α(n)) ≈ O(n)   α = inverse Ackermann, effectively constant
Dijkstra             O((V + E) log V)    non-negative weights ONLY
```

Graph Representation

```java
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);
}
```

Core Templates

Variables: queue = BFS frontier · visited[] = seen flags · level = current distance ring · inDegree[] = remaining prerequisites per node · order = topo result · parent[]/rank[] = union-find forest · dist[] = shortest distance · pq = min-heap on cost · color[] = 2-coloring (-1/0/1)

Greedy

Two Interval Templates

```
MERGE DIRECTION RULE (for merge-style problems like #56, where both keys work):
    Sort by START  →  traverse FORWARD  (i = 0 → n−1), extend the END of last kept
    Sort by END    →  traverse BACKWARD (i = n−1 → 0), extend the START of last kept

TEMPLATE 1 — Sort by END time, sweep forward, track end of last kept interval
TEMPLATE 2 — Sort by START time, sweep forward, merge when overlapping
```

Variables: intervals = list of [start, end] · lastEnd = end of last kept interval · result = merged output · pq = min-heap of active end times

Dynamic Programming

All Six Templates

Variables: dp[i] = the answer for the subproblem at index/state i (meaning varies per family: ways/cost/length/feasibility ending at or reaching i) · n = problem size · target/j = knapsack capacity dimension.

Pseudocode:

```
LINEAR 1D:   dp[i] = fixed recipe over dp[i-1], dp[i-2] (Kadane: best ending at i = max(start fresh, extend))
LOOK-BACK:   for each i, scan all earlier j and extend the best dp[j]
KNAPSACK:    collapse item dimension to 1D; descending j = each item once, ascending j = reusable
2D SEQUENCE: match consumes both strings (diagonal), mismatch drops one side
INTERVAL:    solve short ranges first, combine via a split point inside the range
GRID:        each cell accumulates from its up/left neighbours
STATE MACHINE: name a few states, update each from yesterday's states every step
```

Variables: dp[j] = number of ways to reach sum j (count flavor; dp[0] = 1 is the empty selection) · i = item index · j = current capacity/target, num = an item's weight/value. Loop direction decides reuse: descending j = item used at most once, ascending j = item reusable; target-outer = orderings counted.

```
0/1 KNAPSACK (each item once):
    dp[0] = 1
    for each item i:
        for j from target DOWN TO nums[i]:
            dp[j] += dp[j - nums[i]]

UNBOUNDED KNAPSACK (item reusable):
    dp[0] = 1
    for each item i:
        for j from nums[i] UP TO target:
            dp[j] += dp[j - nums[i]]

PERMUTATION COUNT (order matters):
    dp[0] = 1
    for j from 1 to target:
        for each num in nums:
            dp[j] += dp[j - num]
```

Variables: dp[i][j] = the answer for the prefixes s1[0..i) (first i chars) and s2[0..j) (first j chars); index 0 = empty prefix.

Variables: dp[i][j] = the answer for the interval [i, j] (left endpoint i, right endpoint j).

Variables: dp[i][j] = the count/answer for the (i, j) subproblem, defined only for j ≤ i (lower-triangular).

Variables: dp[i][j] = best (path count / cost) to reach cell (i, j) moving only right/down; memo[i][j] = longest path starting from (i, j) for the all-direction DFS variant; dr/dc = 4-direction deltas.

```
dr = {1,-1,0,0}; dc = {0,0,1,-1}
// DOWN UP RIGHT LEFT
```

Bit Manipulation

Canonical Template

Variables: result = XOR accumulator (lone survivor) · n = number being inspected/edited · i = bit position · count = set-bit counter · mask = bit-set letter-presence set · masks[] = selects bit in x · & (n−1) clears lowest set bit; n & (−n) isolates lowest set bit

```
XOR CANCEL:
    result = 0
    for each num in nums: result = result XOR num   (pairs cancel, lone survives)

COUNT SET BITS (Brian Kernighan):
    count = 0
    while n != 0: count = count + 1; n = n & (n-1)   (each step removes one set bit)

BITMASK AS SET:
    mask = 0
    for each char c in word: mask = mask | (1 << (c - 'a'))
    two words share no letter when (mask[i] & mask[j]) == 0
```

Variations

Variables: ones/twos = bits seen 1 / 2 times mod 3 · xor = a ^ b of the two uniques · diff = lowest differing bit (xor & −xor) · a/b = the two unique values · dp[] = set-bit counts by number · shift = bits dropped to reach common prefix · carry = AND-derived carry bits

```
VARIATION 1 (mod-3 state machine, Single Number II):
    ones = 0; twos = 0
    for each num: ones = (ones XOR num) AND NOT twos; twos = (twos XOR num) AND NOT ones

VARIATION 2 (split XOR into two groups, Single Number III):
    xor = XOR of all nums
    diff = xor AND (-xor)     (lowest bit where a and b differ)

VARIATION 3 (DP relation, Counting Bits):
    dp = array of size n+1
    for i = 1..n: dp[i] = dp[i >> 1] + (i AND 1)

VARIATION 4 (common prefix, Bitwise AND of Range):
    shift = 0
    while left != right: left >>= 1; right >>= 1; shift = shift + 1
    return left << shift

VARIATION 5 (carry loop, Sum of Two Integers):
    while b != 0:
        carry = a AND b
        a = a XOR b
        b = carry << 1
```

Design

Canonical Template — HashMap + Doubly Linked List (LRU)

Variables: map = key → CacheNode · head/tail = sentinels (head side = MRU, tail side = LRU) · node.previous/node.next = doubly linked neighbors

Pseudocode:

```
CacheNode: key, value, previous, next
init: head <-> tail (head.next = tail; tail.previous = head)

remove(node):
    node.previous.next = node.next
    node.next.previous = node.previous

insertAfterHead(node):        # mark as most-recently-used
    link node between head and head.next

evict LRU:
    lru = tail.previous        # node just before tail
    remove(lru); map.remove(lru.key)
```

```java
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int value) { this.key = key; this.value = value; }
}
```

Variations

Variables: keyToVal/keyToFreq = key lookups · freqToKeys = freq → ordered keys at that freq · minFreq = smallest live frequency (RandomizedSet) list = values array · map = val → index

Math

Canonical Templates

Variables: n = the number being processed · digit = last digit peeled · x = base in fast power · result = running product/accumulator · b = the base for conversion · composite[] = sieve marks

```
DIGIT EXTRACTION:
    while n is not zero:
        digit = remainder of n divided by 10
        drop the last digit of n
        use digit

FAST POWER (x^n):
    result starts at 1
    while exponent n still has bits:
        if the lowest bit is set, fold the current x into result
        square x to reach the next power
        shift n right to consume that bit
    return result

BASE CONVERSION (number -> digits):
    while num is positive:
        append num mod b
        divide num by b
    reverse the collected digits

BASE CONVERSION (digits -> number):
    value starts at 0
    for each digit: value = value * b + digit

SIEVE:
    make a composite[] flag array up to n
    for i from 2 while i*i <= n:
        if i already marked composite, skip it
        mark every multiple of i starting at i*i as composite
```